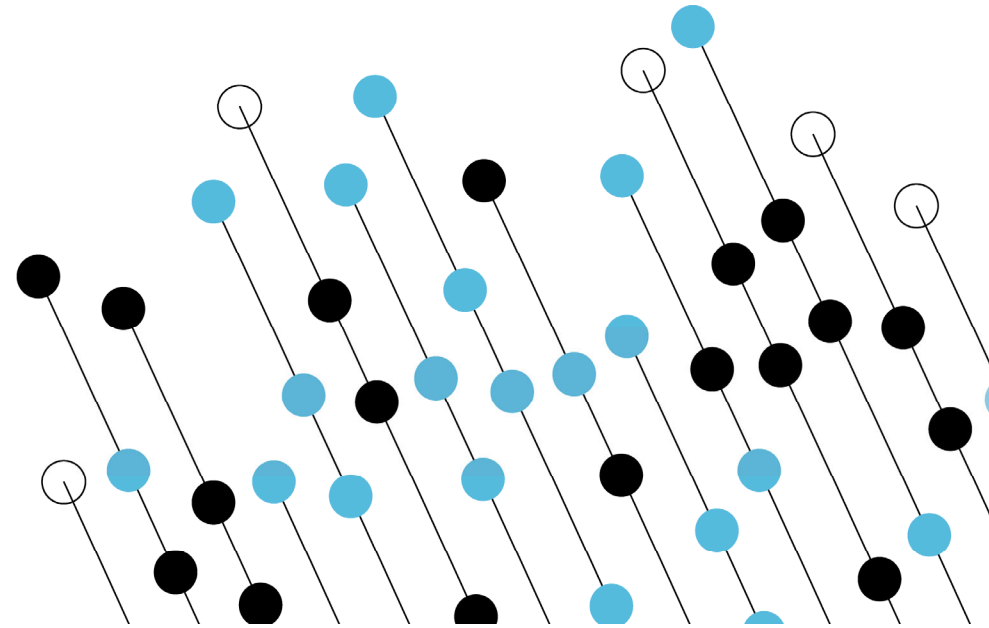


Inheritance



Contents



Objectives

- To add functionality to existing classes using inheritance.



Contents

- Basic concepts of inheritance.
- Extending a simple class.



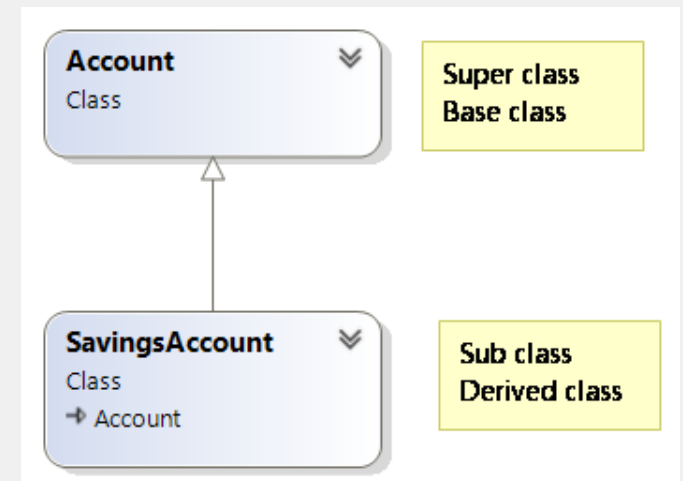
Hands-on labs

Base and derived classes

- A class can inherit the features of another class.
- The 'sub' class can:
 - Utilise all the features of the super class.
 - Override certain behaviour of the super class.
 - Add new features.
- Inheritance is a fundamental object-oriented concept.

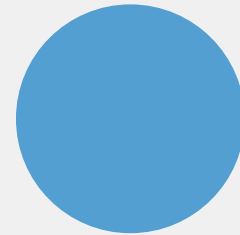
The code in the super class can be reused by the subclass. ✓

New classes can be defined simply in the terms of their **differences** from an existing class. ✓

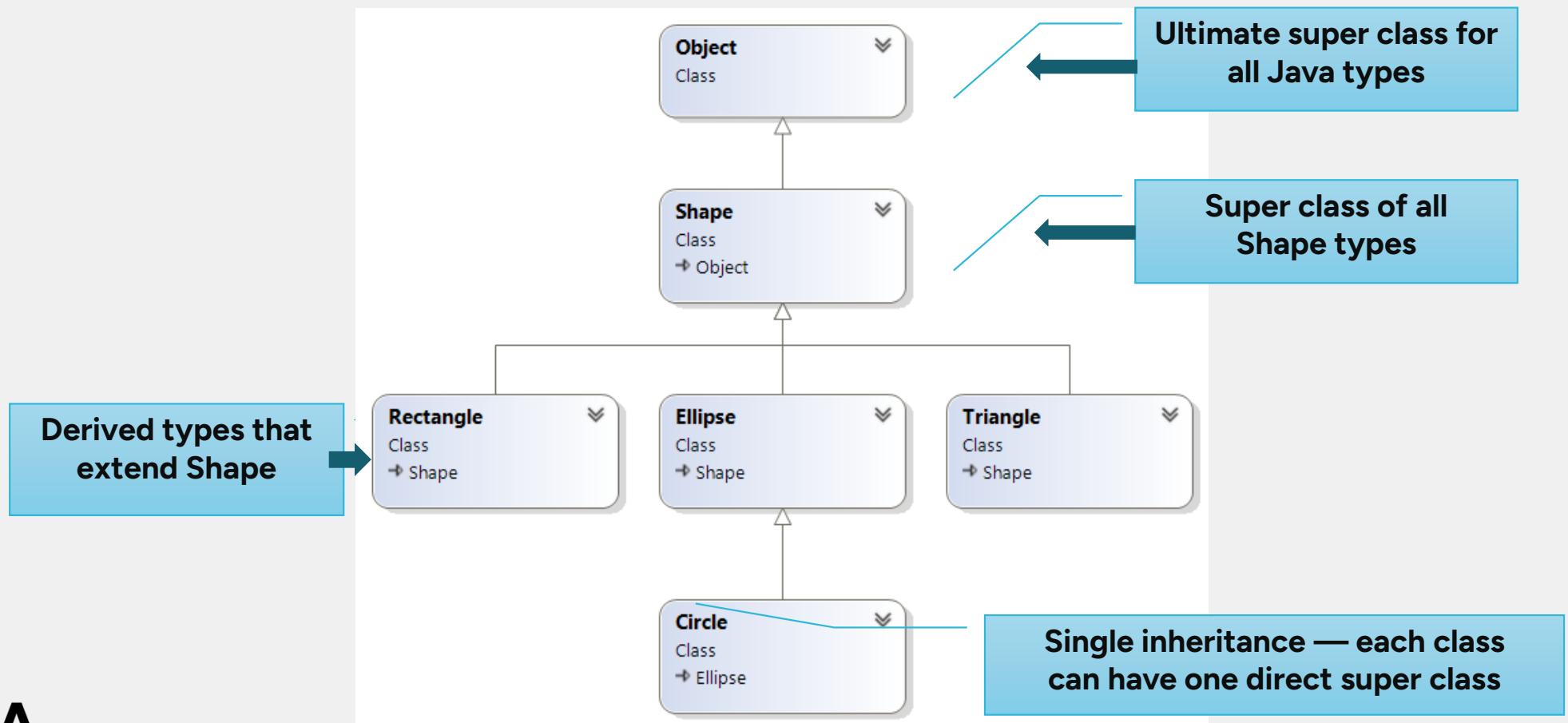


Inheritance in action

- **A vector graphics program:**
 - Lots of commonality
 - position and colour fields
 - draw method
 - Want to benefit from re-use
- **Create a base class called Shape:**
 - Implement common code there
- **Derive classes from Shape:**
 - Rectangle, Ellipse, Triangle



The inheritance hierarchy



Specifying the base class

Super
class

```
public class Shape {  
    private int x, y;  
    private String colour;  
    ...  
}
```

Sub classes
extending the
super class

```
public class Rectangle extends Shape {  
    ...  
}
```

Rectangle-specific members

```
public class Ellipse extends Shape {  
    ...  
}
```

Ellipse-specific members

```
public class Circle extends Ellipse {  
    ...  
}
```

Circle-specific members

Sub class inherits all the super class fields

Can be exposed
via public
getters

```
public class Shape {  
    private int x, y;  
    private String colour;  
    ...  
}
```

```
public class Ellipse extends Shape {  
    private int width;  
    private int height;  
    ...  
}
```

X & y:	10, 10
colour:	Green

Shape object

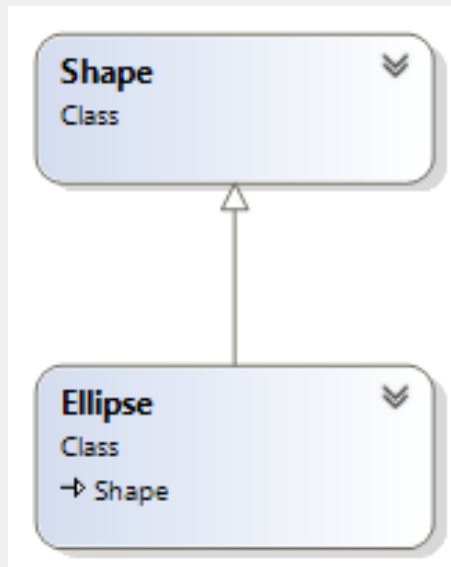
X & y:	10, 10
colour:	Green
width:	20
height:	10

Ellipse object

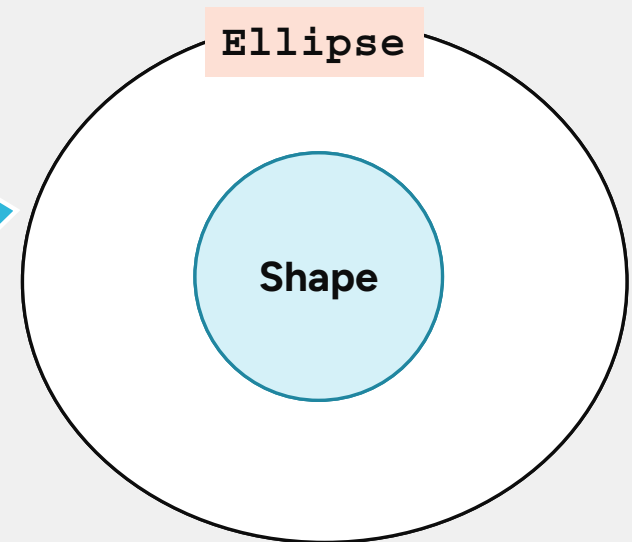
A sub type is a kind of super type

Constructing the derived objects

- **Base class constructors are not inherited.**
 - But default constructor of the base class is called.
- **You can invoke base class constructor.**
 - This is **mandatory** if there is no default (no argument) constructor in the base class.



Before constructing an Ellipse,
you should construct the Shape
object within.



Derived class constructor

```
class Shape {  
    private int x, y;  
    private String colour;  
  
    public Shape(int x, int y, String colour) {  
        this.x = x; this.y = y;  
        this.colour = colour;  
    }  
}
```

No default constructor, so all derived classes must invoke this constructor

```
class Ellipse extends Shape {  
    private int width, height;  
  
    public Ellipse(int x, int y, int width, int height, String colour) {  
        super(x, y, colour);  
        this.width = width;  
        this.height = height;  
    }  
}
```

Calling base constructor to initialise base fields

```
Ellipse e1 = new Ellipse(4,7, 23, 24, "RED");
```

Derived class constructor — Chaining

```
class Ellipse extends Shape {  
    private int width, height;  
  
    public Ellipse(int x, int y, int width, int height, String colour) {  
        super(x, y, colour);  
        this.width = width;  
        this.height = height;  
    }  
    public Ellipse(int x, int y) {  
        this(x, y, 10, 10, Color.BLUE);  
    }  
}
```



Calling base constructor initialise base fields

Calling Ellipse constructor

```
Ellipse e1 = new Ellipse(4,7);
```

Calling super class methods

```
public class Account {  
    double balance;  
    int id;  
    public Account(int id) {  
        this.id=id;  
    }  
  
    public String getCharacteristics() {  
        return "Account details here...";  
    }  
}
```

```
public class SavingsAccount extends Account{  
    public SavingsAccount(int id) {  
        super(id);  
    }  
  
    public String getCharacteristics() {  
        return super.getCharacteristics() +  
            "SavingsAccount details";  
    }  
}
```



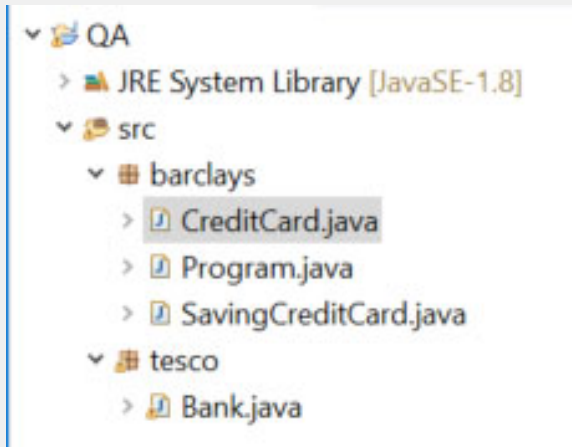
Can call **super**.method on any line

Protected

- **Modifier that allows access to deriving types in any package:**
 - Used to restrict access to methods
 - Fields should always be private
- **Let's view an example...**



Protected access modifier



```
package tesco;
import barclays.*;
public class Bank {
    public static void main(String[] args) {
        CreditCard cc = new CreditCard(333);
        System.out.println(cc1.pin);
    }
}
```

✗

```
package barclays;
```

```
public class CreditCard {
    protected int pin;
```

```
    public CreditCard(int pin) {
        this.pin = pin;
    }
}
```

Can be accessed anywhere
in the same package

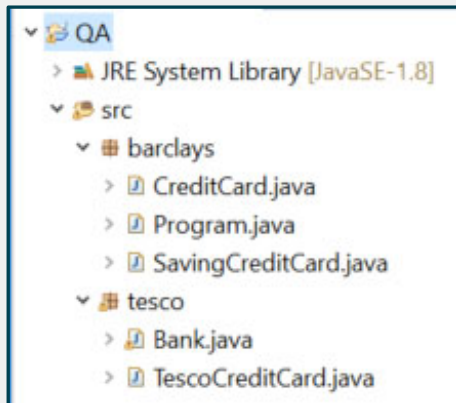
```
package barclays;
```

```
public class Program {
    public static void main(String[] args) {
        CreditCard cc1 = new CreditCard(111);
        System.out.println(cc1.pin);
    }
}
```

✓

But not by a class outside of
the package

Protected example within inheritance



```
package barclays;

public class CreditCard {
    protected int pin;

    public CreditCard(int pin) {
        this.pin = pin;
    }
}
```

```
package tesco;
import barclays.CreditCard;

public class TescoCreditCard extends CreditCard {
    public TescoCreditCard(int pin) {
        super(pin);
    }
    public void changePin(int newPin) {
        this.pin = newPin;
    }
}
```



Can be accessed by a class in another package which extends the class

Using inheritance for creating custom exceptions

- **Custom exception class must derive from `Exception`:**
 - Duplicate constructors.
 - Pass 'String message' up to base class (the only time you can write to the inherited message field).
 - Can add additional methods.
- **View code example on the next slide...**



Java example

```
public class QAException extends Exception {  
  
    public QAException(String message) {  
        super(message);  
    }  
    public QAException() {  
        super("General error");  
    }  
  
    public void log() {  
        // log the message field  
    }  
}
```

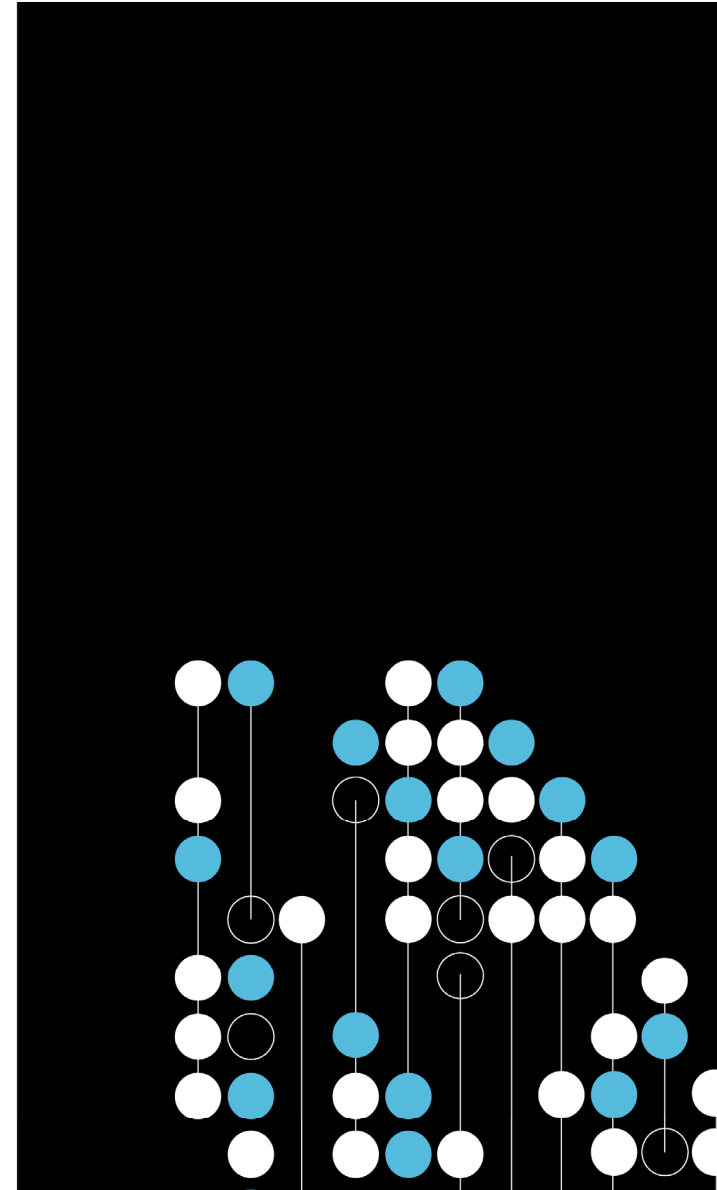
```
void methodY() throws QAException {  
  
    throw new QAException();  
}
```

```
void methodX() {  
    try {  
        methodY();  
    } catch (QAException e) {  
        System.out.println(e.getMessage());  
        e.log();  
    }  
}
```



Review

- **Why do we do inheritance?**
 - Code reuse.
 - Perhaps there will be other reasons soon!
- **Sub class inherits and can add additional functionality.**



Lab

- Working with inheritance.

Duration: 90 minutes.

QA

